

Обучение глубоких нейронных сетей

Обучение глубоких нейронных сетей – это процесс настройки весов сети с использованием большого объёма данных для минимизации ошибки предсказания. Ключевым алгоритмом обучения является метод обратного распространения ошибки (backpropagation), который вместе с градиентным спуском оптимизирует веса, улучшая результаты сети постепенно.

Глубокие нейронные сети состоят из нескольких слоев нейронов, где каждый слой преобразует входные данные. Обучение заключается в том, чтобы сеть училась на ошибках: при каждом проходе по данным рассчитывается ошибка предсказания, затем её градиенты по весам передаются обратно, и веса корректируются с помощью градиентного спуска. Это цикл повторяется много раз (эпох), пока ошибка не станет минимальной или сеть не перестанет улучшаться.

Градиентный спуск – основной алгоритм, который обновляет веса сети, двигаясь по направлению наискорейшего уменьшения функции ошибки. Параметр "скорость обучения" (learning rate) определяет размер шага обновления.

Обратное распространение ошибки – эффективный способ вычисления градиентов для каждого веса сети, позволяющий применять градиентный спуск к многослойным моделям.

Технические сложности и улучшения

Глубокие сети могут быть сложны для обучения из-за эффекта затухающего градиента и высоких вычислительных затрат. Для улучшения обучения применяют такие техники, как предварительное обучение слоёв, расширенные алгоритмы оптимизации, регуляризация, а также современные методы вроде смешанной точности и дистилляции моделей.

Проблемы исчезания и взрывного роста градиентов – ключевые трудности при обучении глубоких нейронных сетей.

Исчезновение градиентов

Это явление возникает, когда значения градиентов при обратном распространении ошибки становятся очень малыми (экспоненциально уменьшаются) по мере продвижения к начальным слоям сети. В результате веса первых слоев обновляются незначительно или почти не меняются, что затрудняет обучение и мешает выявлению сложных признаков. Причинами являются особенности функций активации (например, сигмоида) и умножение на маленькие веса в глубоких слоях. Особенно остро эта проблема проявляется в рекуррентных нейронных сетях (RNN), затрудняя обучение долгосрочных зависимостей. Для борьбы с этим используют функции ReLU, остаточные связи (ResNet), специальные инициализации весов и отсечение градиентов.

Взрывной рост градиентов

Противоположная проблема – когда значения градиентов становятся очень большими при обратном проходе, что вызывает чрезмерно большие обновления весов. Это приводит к нестабильности обучения, резким скачкам значений параметров и может приводить к расходимости модели. Взрывные градиенты часто возникают из-за больших начальных весов или накопления производных, превышающих 1. Для предотвращения применяются техники отсечения градиента (gradient clipping) и регулирование величины шагов обновления

Рассмотрим пример создания нейросети для умножения числа на 3.

Создадим нейронную сеть, состоящую из одного нейрона.



```
from keras.layers import Dense
from keras.models import Sequential

model = Sequential([
    Dense(1, input_shape=(1,), activation='relu')
])

model.summary()
```

Sequential – это класс последовательности слоев в нейронной сети, у нас пока будет только один слой, но сюда можно добавлять сколь угодно слоев и это будет сеть, состоящая из последовательности слоев.

Dense – это класс полносвязного/линейного слоя, все нейроны связаны друг с другом.

units=1 – это количество нейронов в слое. В нашем случае 1 нейрон.

input_shape=(1,) – это входная размерность объекта. В нашем случае только 1 вход.

activation='relu' – это функция активации, которая добавляет в слой нелинейности, именно из-за неё можно получать более сложные результаты работы сети.

Для того, что обучить нейронную сеть для любой задачи нужно ответить на три вопроса:

1. Какая архитектура сети?

2. Что оптимизируем?

3. Как обучаем?

1. Какая архитектура сети?

Сеть состоит из одного слоя и одного нейрона.

```
from keras.layers import Dense
from keras.models import Sequential

model = Sequential([
    Dense(1, input_shape=(1,), activation='linear')
])

model.summary()
```

 Model: "sequential_7"

Layer (type)	Output Shape	Param #
dense_7 (Dense)	(None, 1)	2

Total params: 2 (8.00 B)
Trainable params: 2 (8.00 B)
Non-trainable params: 0 (0.00 B)

]

```
w1, w0 = model.get_weights()
w1, w0
```

 (array([[1.6241895]], dtype=float32), array([0.], dtype=float32))

2. Как оптимизируем?

Оптимизатор обновляет параметры модели, за счет чего минимизируется функция ошибки. Существует несколько видов оптимизаторов, например, градиентный спуск – это один из методов оптимизации для нейронной сети.

```
model.compile(optimizer='sgd', loss='mse', metrics=['mae'])
```

3. Как обучаем?

Для обучения указываем количество эпох, то есть сколько полных проходов будет сделано по обучающему датасету

```
model.fit(X, y, epochs=100)
```

```

1/1 ————— 0s 38ms/step - loss: 5.5815 - mae: 1.9654
Epoch 3/100
1/1 ————— 0s 42ms/step - loss: 0.4829 - mae: 0.5447
Epoch 4/100
1/1 ————— 0s 36ms/step - loss: 0.0504 - mae: 0.1743
Epoch 5/100
1/1 ————— 0s 57ms/step - loss: 0.0136 - mae: 0.1074

```

Видно, что сеть обучается, ошибка падает, метрика становится лучше. Проверим, работу модели на новых данных.

```

user_inp1, user_inp2 = 5, -9
print(f"Проверка на новых данных: {user_inp1} {user_inp2}")
print("Предсказание нейронной сети: ")
print(model.predict(np.array([[user_inp1], [user_inp2]])))

```

```

Проверка на новых данных: 5 -9
Предсказание нейронной сети:
1/1 ————— 0s 37ms/step
[[ 15.030944]
 [-26.7396  ]]

```

И проверим на всех обучающих данных.

```

import pandas as pd

pd.DataFrame({
    'true': np.squeeze(y),
    'pred': np.squeeze(model.predict(X))
})

```

```

1/1 ————— 0s 43ms/step

```

	true	pred
0	3	3.096503
1	9	9.063724
2	6	6.080113
3	30	29.948994
4	12	12.047334
5	21	20.998163
6	24	23.981773

Сеть для сложения чисел

Создадим обучающие данные

```
X1 = np.random.randint(1, 10, size=50)
X2 = np.random.randint(1, 10, size=50)
```

```
y = X1 + X2
```

```
X = np.vstack([X1, X2]).T
X
```

```
array([[3, 7],
       [7, 1],
       [3, 1],
       [5, 4],
       [6, 3],
       [3, 2],
       [8, 9],
       [7, 3],
       [6, 2],
       [4, 5],
       [1, 3],
       [7, 4],
```

```
y = y[None]
y = y.T
y
```

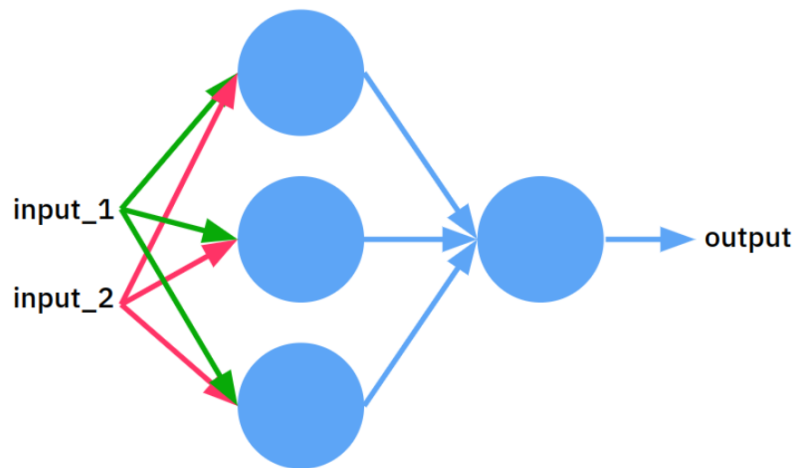
```
array([[10],
       [ 8],
       [ 4],
       [ 9],
       [ 9],
       [ 5],
       [17],
       [10].
```

```
from sklearn.preprocessing import MinMaxScaler
```

```
mms = MinMaxScaler()
X_norm = mms.fit_transform(X)
```

1. Какая архитектура сети

Создадим сеть посложнее, она будет состоять уже из двух слоев. В одном слое 3 нейрона, а в выходном слое 1 нейрон.



При этом пользуемся Sequential, т.к. у нас слои последовательные, сначала 2 входа идут в 5 нейронов, а затем выходы из этих 5 нейронов идут в один нейрон.

```
from keras.layers import Dense
from keras.models import Sequential
tf.random.set_seed(9)

model = Sequential([
    Dense(3, input_shape=(2,), activation='linear'),
    Dense(1, activation='linear')
])

model.summary()
```

Что здесь происходит с весами и с самой архитектурой? Количество весов для одного нейрона равно 2, т.к. два входа поступает, а вдобавок у каждого нейрона есть 1 bias, на первом слое 3 нейрона, значит суммарно весов на первом слое будет 9. Второй слой ждет на вход 3 сигнала, еще 1 bias. Получаем $9 + 4 = 13$ обучаемых весов.

```
model.get_weights()
```

```
[array([[ -0.8020328 , -0.19388759,  0.68648016],
        [ 0.47121418, -0.38612115,  0.52383924]]), dtype=float32),
 array([0., 0., 0.], dtype=float32),
 array([[ -1.0760131 ],
        [ 0.22256064],
        [-0.3608055 ]], dtype=float32),
 array([0.], dtype=float32)]
```

Что оптимизируем?

Снова задача регрессии, поэтому берем функцию потерь MSE.

Как оптимизируем?

Возьмем тот же самый градиентный спуск со стохастикой.

```
model.compile(optimizer='sgd', loss='mse', metrics=['mae'])
```

```
%%time
```

```
model.fit(X_norm, y, epochs=200)
```

```
Epoch 1/200
2/2 ————— 1s 18ms/step - loss: 118.1128 - mae: 10.1310
Epoch 2/200
2/2 ————— 4s 4s/step - loss: 87.2784 - mae: 8.5860
Epoch 3/200
2/2 ————— 2s 748ms/step - loss: 55.0206 - mae: 6.6001
Epoch 4/200
2/2 ————— 0s 8ms/step - loss: 24.8597 - mae: 4.0707
Epoch 5/200
2/2 ————— 0s 10ms/step - loss: 9.0502 - mae: 2.4844
Epoch 6/200
2/2 ————— 0s 7ms/step - loss: 5.1627 - mae: 1.9364
```

Сеть обучается, ошибка падает, метрика становится лучше. Теперь проверим, как модель работает на новых данных.

```
| test_X = [[4, 2],
            [6, 2]]
test_X = mms.transform(test_X)
print("Предсказание нейронной сети: ")
print(model.predict(np.array(test_X)))
```

```
· Предсказание нейронной сети:
1/1 ————— 0s 45ms/step
[[6.0012817]
 [8.000473 ]]
```

Предсказания очень похожи на истину. Проверимся на всех обучающих данных.

```
import pandas as pd
```

```
pd.DataFrame({
    'x1': X[:, 0],
    'x2': X[:, 1],
    'true': np.squeeze(y),
    'pred': np.squeeze(model.predict(X_norm))
}).head(10)
```

	x1	x2	true	pred
0	3	7	10	10.000703
1	7	1	8	8.000266
2	3	1	4	4.001883
3	5	4	9	9.000484
4	6	3	9	9.000277
5	3	2	5	5.001687